

Predicting the unpredictable with Bayesian Inference: Real time water demand

Nick Belement

March 2025

Abstract

Forecasting long sequences of time series data plays a critical role across a wide range of domains, including financial market analysis, energy grid optimization, climate modelling, and pharmaceutical development. In the context of water resource management, accurate demand forecasting enables utilities to maintain operational efficiency, reduce waste, and respond to future variability in supply and demand. This study investigates the integration of Bayesian inference techniques with Transformer-based neural architectures to model real-time water demand, specifically using Monte Carlo (MC) Dropout as a lightweight and scalable approach for quantifying uncertainty. The results show that the Bayesian Informer model not only achieves strong predictive performance but also provides calibrated uncertainty estimates that help diagnose periods of high forecast risk. The ability to capture and communicate uncertainty is particularly beneficial in high-stakes environments like urban water networks, where over- or underestimating demand can have costly consequences. Overall, this study highlights the importance of uncertainty quantification with deep learning models in infrastructure decision-making.

1 Introduction

The UK water industry faces varying levels of crisis in today’s climate. Since its privatisation under Thatcher’s Water Act in 1989, the under-invested industry has been flagged with huge concerns in regards to excessive accumulated debts; especially, Thames Water which has piled a staggering debt of £18bn. These alarms even faced potential prospects of a renationalisation last year. As of present, water companies provide roughly 14 billion litres of water a day with expected increase of an additional 4 billion by 2050 from climate and supply-demand gap pressures. [1] For the UK to pave way towards a sustainable water future, the following challenges have to be addressed: Population growth, Ageing Infrastructure & Climate change. With increased funding into digital water networks, AI implementation with real-time data has allowed for more informed decision-making within the sector. In recent years, the integration of machine learning (ML) into urban water management systems has gained significant momentum, driven by the increasing availability of high-frequency sensor data and the pressing need for adaptive, real-time decision-making. Traditional models often fall short in tasks that require learning of complex dynamic patterns present in such volatile data. This has led to a paradigm shift toward data-driven approaches capable of learning intricate relationships from historical patterns and external covariates without relying on explicit rule-based formulations. Sequential models offer a structural advantage by learning from the temporal continuity and are particularly adept at identifying subtle fluctuations inherent in time series data. In the context of water utilities, the ability to anticipate fluctuations with minimal latency can support more efficient resource allocation, reduce the risk of supply-demand mismatches, and ultimately contribute to more resilient infrastructure management. Despite this, deep learning models face new challenges; the need for large volumes of clean, labelled data, susceptibility to overfitting and difficulty in quantifying uncertainty. In high-stakes applications such as water resource management, where under-prediction can cause shortages and over-prediction leads to wasteful overproduction, the ability to quantify and trust predictions is as important as the accuracy itself. Combining machine learning with Bayesian inference techniques allows models to express predictive uncertainty through posterior distributions. In this study, I investigate the effectiveness of modern sequential architectures, specifically the Transformer and Informer, for short to medium term water demand forecasting; and explore how Monte Carlo Dropout enhances the reliability of forecasts. By benchmarking each architecture across multiple forecasting horizons, I explore the trade-offs between model complexity, interpretability, and real-world applicability in the context of urban water demand forecasting.

2 Background & Data

Accurate forecasting of water demand is a critical component of sustainable water resource management, ensuring that supply meets fluctuating demand without excess energy use or environmental impact. Water demand refers to the total volume of water needed to supply consumers or businesses within a certain range of time. Distribution Input (DI) is the total volume of treated water put into the water supply network from a Water Treatment Work (WTW) over a given time-frame.

Utility providers must ensure there is sufficient water treated and distributed to meet consumption, while avoiding overproduction, resource waste, and energy inefficiencies. This balancing act is especially critical in today’s stride of understanding climate variability within the context of urbanisation. One of the most widely monitored metrics by water companies is the DI and forms the basis for many operational decisions in the water industry, including pumping schedules, treatment chemical dosing, and leakage detection thresholds.

This study focuses on the Restormel Water Treatment Works, located near Bodmin in Cornwall. It is a cornerstone facility in South West Water’s network, supplying potable water to much of Cornwall’s residential population which has a substantial influx of tourists during the summer months. Built in 1960 and subsequently upgraded through multiple capital programs, Restormel WTW has a treatment capacity of up to 200 megalitres per day (ML/d), with visibly more output in the Summer than during Winter, highlighting the clear influence of external drivers such as weather and time - making it an ideal case study for the time series problem at hand. As part of the company’s broader regulatory compliance and infrastructure investment under AMP5, the Restormel site underwent significant modernisation to ensure long-term reliability and quality[2], including the integration of granular activated carbon (GAC) contactors and enhanced automation systems. These improvements not only support water quality and continuity of supply but also increase the richness and reliability of operational data available for forecasting.

I obtained DI data for eight different WTWs in the South West Waterwork network for the previous year, 2024. The original time series dataset consisted of 35,138 timestamped records with a rich granularity of 15 minute intervals to reflect the temporal patterns and long term consumer trends. Each WTW was recorded across multiple columns, with their respective site categorised by the ‘High Integrity Pressure Supply’ (HIPS), which corresponds to different sub-zones within the water network. These zones are engineered to maintain consistent

supply pressure in geographically elevated or isolated regions.

While not all columns were suitable for forecasting due to sparsity or noise, they reflect the operational complexity of real-world water systems and offer opportunities for future multi-output forecasting models.

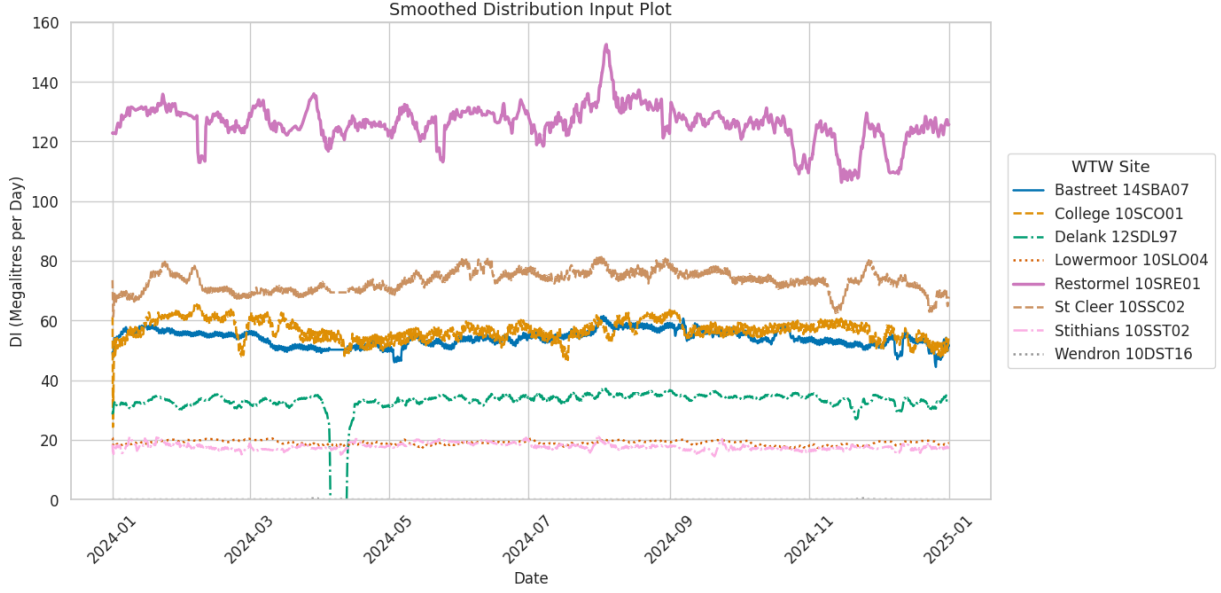


Figure 1: Half-week smoothed daily water demand for selected WTWs in 2024.

I narrowed my focus on the site Restormel 10SRE01 due to its high strategic importance, significant seasonal variability, and the quality of metadata available on the site’s operation. The benefit of having this data with a high temporal resolution allows us to model fine-grained demand dynamics and short term volatility, whilst also being able to identify longer-term seasonal trends when resampled. Figure 1 shows the DI values aggregated into hourly means to reduce noise, and smoothed using a 84 hour (half-week) rolling average to provide clarity into the overall trend of water demand throughout 2024.

In order to capture the temporal dynamics of water demand, a key part of the modelling pipeline involved constructing a set of time-dependent features from the raw DI series. These features therefore help encode the periodicity, trends, and external temporal context of water usage patterns for enabling neural networks to learn from sequential data. The following features were derived:

- **Hour of the Day** ($h \in \{0, 1, \dots, 23\}$): for capturing intraday variation for human activity cycles.
- **Day of the Week** ($d \in \{0, \dots, 6\}$): capturing weekly seasonality between weekdays and weekends.
- **Month of the Year** ($m \in \{1, \dots, 12\}$): models longer seasonal trends such as winter drops or summer peaks in usage.
- **Weekend**: a binary flag indicating weekend periods, associated with non-working days and different usage profiles.

Cyclical transformations using sine and cosine functions were applied to ensure that these features were encoded properly, otherwise for example 23:00 and 00:00 would appear distant in linear if not for the appropriate transformation resulting in higher context-aware sensitivity to human-driven consumption patterns.

Whilst my dataset contained richness in terms of temporal resolution and seasonality, one key challenge I encountered during preprocessing was the presence of irregular gaps and outliers within the DI readings. I first began exploring Kalman smoothing, a common method used in time-series signal reconstruction. Part of a broader family of state-space models that represent time series’ as output of underlying systems evolving through latent (hidden) states over time, they are suitable when the system is assumed to follow linear Gaussian dynamics. Kalman filtering (KF) operates on a recursive update mechanism that estimates a system’s latent state $\mathbf{x}_t$ at each time step t , given a sequence of noisy observations $\mathbf{y}_{1:t}$. The model assumes the system evolves according to a linear Gaussian state-space model, composed of the following two equations:

- **State Transition (Process Model):**

$$\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q})$$

- **Observation Model:**

$$\mathbf{y}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(0, \mathbf{R})$$

Here, $\mathbf{w}_t$, $\mathbf{v}_t$ represent Gaussian process and observation noise and matrices, $\mathbf{A}$ and $\mathbf{H}$, define the system’s transition and observation structure, while $\mathbf{Q}$ and $\mathbf{R}$ encode uncertainty in state evolution and measurement. The Kalman Filter performs two steps recursively:

Prediction (prior):

$$\begin{aligned}\hat{\mathbf{x}}_{t|t-1} &= \mathbf{A}\hat{\mathbf{x}}_{t-1|t-1} \\ \mathbf{P}_{t|t-1} &= \mathbf{A}\mathbf{P}_{t-1|t-1}\mathbf{A}^\top + \mathbf{Q}\end{aligned}$$

Update (posterior):

$$\begin{aligned}\mathbf{K}_t &= \mathbf{P}_{t|t-1}\mathbf{H}^\top (\mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^\top + \mathbf{R})^{-1} \\ \hat{\mathbf{x}}_{t|t} &= \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t (\mathbf{y}_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1}) \\ \mathbf{P}_{t|t} &= (\mathbf{I} - \mathbf{K}_t\mathbf{H}) \mathbf{P}_{t|t-1}\end{aligned}$$

KF’s appeal lies in its recursive nature — where prediction is followed by a correction — conceptually mirroring online learning or the hidden state updates in neural networks, where the model maintains and updates a memory representation of temporal history. The Kalman gain $\mathbf{K}_t$, in a sense, plays a similar role to an attention weight, balancing prior belief with new evidence. This interpretability makes Kalman filters particularly appealing in fields where transparency and physical plausibility are valued.

While Kalman smoothing offers a mathematically elegant way of handling noisy sequences, missing or outlier values of DI remains a limitation. These realisations in the dataset weren’t always randomly distributed gaps in telemetry yet correspond to genuine operational anomalies. This could be: scheduled maintenance of pumps, deliberate flow suppression for asset inspections, cleaning operations in treatment works or even temporary shutdowns. All of these cases pose barriers for statistical modelling, with KF essentially filling in gaps under assumptions of continuity. In the case of operational anomalies, the missingness of our data therefore becomes informative. It’s not flagged as ‘missing at random’ (MAR), but rather falls under ‘missing not at random’ (MNAR), where the fact that a value is missing conveys information about the system itself. My choice of retaining anomalously low DI values to capture the underlying real-world complexity of water demand. It is often perhaps more valuable to accurately characterise uncertainty — including recognising what is unknown — than to chase perfect point estimates. As such, the handling of missing data is not simply a preprocessing step, but a core part of the model’s epistemology and adds for further discussion on this work.

3 Literature Review

Forecasting water demand in real-time presents us with long-standing challenges in the intersection of environmental sciences and machine learning. As water utility stations transition towards digital infrastructure and smart metering, there is an increasing need to develop models that not only predict consumption accurately but also account for structural changes in the network, correlated spatial patterns, and incomplete data sources. Gil-Gamboa et al. (2024)[3] proposed a comprehensive methodology for medium-term water consumption forecasting; combining preprocessing, customer clustering, and deep feed-forward neural networks (DFFNNs) to model residential water demand. They highlighted a key limitation of many real-world water datasets—namely, their heterogeneity and noise due to metering failures, consumption anomalies, and differences in user profiles. By using k-means clustering to isolate residential consumers from institutional and industrial users, their forecasting model generalised behavioural patterns greatly. Their results proved that deep neural networks outperform traditional machine learning methods of random forests (RFs) and support vector machines (SVMs) in quarterly-level forecasting tasks, underscoring the value of deep learning for pattern recognition in water use behaviour.

Complementing this, Malialis et al. (2024) [4] addresses short-term demand forecasting within urban water distribution networks (WDNs) by considering spatial correlations between District Metered Areas (DMAs). Their deep learning model forecasts water consumption by combining both a DMA’s historical data and that of highly correlated DMAs, which are identified through Pearson correlation analysis—a statistical technique that measures the linear relationship between two time series by computing the Pearson correlation coefficient, r , ranging from -1 (perfect negative correlation) to $+1$ (perfect positive correlation). This then incorporates information from neighbouring DMAs whose consumption patterns vary synchronously, thereby improving predictive accuracy even when data from the target DMA is incomplete or noisy.

Such approach aligns well with modern utility network design, where resource optimisation often requires cross-zonal awareness. Furthermore, their framework contributes to the emerging field of data-driven digital twins, where models are used not only for forecasting but also for real-time control, anomaly detection, and infrastructure planning.

Building on the need for operationally relevant forecasting solutions, Creaco et al. (2023) presented a novel approach in collaboration with Thames Water for building forecasting models tailored to individual customer profiles. [5] The authors applied feature engineering techniques such as moving averages and lagged values, alongside regularisation to avoid overfitting on spurious noise. Their hybrid forecasting pipeline combined autoregressive (AR) features with machine learning regressors (gradient boosting and ensemble trees), producing stable short-term forecasts without requiring deep models. Although their framework did not rely on complex neural networks, the study emphasised model interpretability, a crucial concern for utility companies deploying forecasts for customer billing and operational planning. Heavy emphasis is made on customer-level granularity, stating that it 'adds an important dimension to the forecasting landscape' — moving from aggregate or DMA-level predictions to individual household behaviours.

4 Experimental Design

4.1 Classical methods

Although limited in their capabilities, it is important to recognise some of the classical statistical models used in the past for forecasting and to acknowledge how machine learning has progressed for such domain. Their purpose lies in the fact that they are the fastest and simplest to interpret. One such method is the seasonal naive forecast, which makes the straightforward assumption: a value at a given time is equal to that observed in the corresponding previous season. Repetition across weeks is as expected highly unlikely thus deeming this method inferior to deep learning models. Following this, another method worth mentioning is the linear AutoRegressive Integrated Moving Average (ARIMA), which models time series data based on its own past values and forecast errors. It is denoted as $ARIMA(p, d, q)$, where p denotes the number of autoregressive (AR) terms, d is the degree of differencing required to make the series stationary, and q corresponds to the number of moving average (MA) terms. In the early stages of my research I extended this to incorporate seasonal components, (SARIMA). The full SARIMA model is expressed as:

$$\Phi_P(B^s) \phi_p(B) (1 - B)^d (1 - B^s)^D y_t = \Theta_Q(B^s) \theta_q(B) \varepsilon_t$$

where $\phi_p(B)$ and $\theta_q(B)$ are the AR and MA polynomials, and $\Phi_P(B^s)$ and $\Theta_Q(B^s)$ are the seasonal components. Despite its simplicity, it's drawbacks are embedded in its linear framework; particularly leading to elevated Root Mean Squared Error (RMSE) and Mean Absolute Percentage Error (MAPE) scores. As seasonal orders increase or multiple seasonalities need to be captured (e.g., daily and weekly cycles), parameter estimation becomes computationally expensive and prone to convergence issues. This became problematic when rolling or real-time forecasts were required, making it unsuitable for high-frequency or long-horizon forecasting at scale. These limitations have contributed to the rise of more expressive and data-driven approaches such as deep learning architectures, which are capable of learning nonlinear dependencies directly from data, adapting dynamically to temporal shifts, and incorporating a wide range of inputs.

4.2 LSTMs

Before the emergence of attention-based models, Recurrent Neural Networks (RNNs) and their variants such as the Long Short-Term Memory networks (LSTMs) were the dominant architectures for modelling sequential data. Designed specifically for time-ordered inputs, LSTMs perform at capturing temporal dependencies by maintaining a memory cell that selectively retains or forgets information across time steps. This allows them to excel in short term to medium range predictions. LSTMs operate by processing sequences one step at a time, iteratively updating a hidden state that encapsulates context of all previous inputs. Its core innovation lies in the gating mechanisms - input, forget and output gates - which regulates the flow of information whilst alleviating the vanishing gradient problem that afflicted vanilla RNNs. The vanishing gradient problem refers to the tendency of gradients becoming extremely small as they are backpropagated through time across many layers or time steps, resulting in earlier inputs having little to no influence on final output. This is why traditional RNNs struggle to learn long-term dependencies. LSTMs ability to retain relevant information over long periods has made them a predominantly safe option in industrial forecasting applications.

Mathematically, an LSTM cell maintains a memory state $\mathbf{c}_t$ and a hidden state $\mathbf{h}_t$ at each time step t , updated through a series of gating mechanisms. Given an input vector $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, the LSTM performs the following operations:

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{(forget gate)} \\
\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{(input gate)} \\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{(cell candidate)} \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{(cell state update)} \\
\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) && \text{(output gate)} \\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{(hidden state update)}
\end{aligned}$$

Here, $\sigma(\cdot)$ denotes the sigmoid activation function, $\tanh(\cdot)$ the hyperbolic tangent function, and $\odot$ the Hadamard (element-wise) product. The forget gate $\mathbf{f}_t$ determines which information from the previous cell state should be discarded, while the input gate $\mathbf{i}_t$ decides what new information to add. The candidate cell state $\tilde{\mathbf{c}}_t$ represents potential updates to the memory, and the cell state $\mathbf{c}_t$ integrates both memory retention and update. Finally, the output gate $\mathbf{o}_t$ modulates the visibility of the memory content through the hidden state $\mathbf{h}_t$. Gradients flowing through $\mathbf{c}_t$ are maintained more stably across many time steps due to the additive nature of the updated cell state, allowing information to persist without exponential decay.

However, LSTMs are not without limitations. Their inherently sequential processing restricts parallelisation across time steps during training, which becomes a bottleneck for long sequences. Moreover, their capacity to model hierarchical or compositional dependencies is bounded by the expressiveness of their memory and gating functions. Despite these limitations, LSTMs remain widely used in real-world forecasting systems due to their proven reliability, interpretability, and the relatively lower data requirements compared to more complex architectures.

Convolutional neural networks (CNNs) are a non-recurrent approach to sequential modelling through applying convolutional operations. Its ability to extract local temporal features using shared filters has garnered traction for its forecasting capabilities, despite being typically associated with image processing tasks. CNNs learn representations by sliding fixed-size kernels over the input data. This sliding window captures local dependencies in a translation-invariant manner, which means the learned filters apply identically across all time steps such that it can detect same patterns regardless of position in the sequence. Additionally, the use of dilated or causal convolutions can extend the receptive field for capturing longer-ranged dependencies without increasing the number of layers or compromising causality. Since CNNs do not require the sequential processing of hidden states, it empowers highly efficient training on data; thus making them an attractive option for real-time applications where low latency and fast inference is critical.

However, by themselves, CNNs struggle to retain global contextual information over extended sequences and so hybrid architectures have emerged as a resolution to such limitations. A widely adopted configuration is the CNN-LSTM model, which first extracts local temporal patterns using convolutional filters and then passes the transformed features into an LSTM layer to capture global sequential dynamics. In this hybrid structure, convolutional layers serve as the front-end feature extractors by scanning the multivariate time series inputs with a set of learnable filters, capturing short-term trends and periodic patterns across different variables. Let $\mathbf{X} \in \mathbb{R}^{T \times d}$ represent the input sequence with T time steps and d features. A 1D convolution operation applied to $\mathbf{X}$ with a kernel $\mathbf{K} \in \mathbb{R}^{k \times d}$ and stride s produces an intermediate representation $\mathbf{Z} \in \mathbb{R}^{T' \times h}$ where:

$$\mathbf{Z}_t = \sigma \left(\sum_{i=0}^{k-1} \mathbf{K}_i \cdot \mathbf{X}_{t+i} + b \right), \quad t = 0, s, 2s, \dots \quad (1)$$

σ is our non-linear activation function, Rectified Linear Unit (ReLU), b is learnable bias, and h denotes the number of filters applied. This operation allows the model to efficiently capture local dynamics and inter-feature interactions across recent history, compressing them into a richer feature map. Following convolution, the feature maps are flattened and temporally aligned before being passed into recurrent layers for global sequence modelling. The LSTM decoder in the hybrid pipeline receives the transformed feature tensor $\mathbf{Z}$ as input, where it processes dependencies over the newly abstracted temporal space. This configuration offers two main benefits: it reduces the sequential length fed to the recurrent layer—thus alleviating training bottlenecks—and it allows the network to learn a hierarchical temporal representation, with the convolutional layer focusing on micro-patterns and the recurrent layer capturing macro-scale transitions. This architectural synergy is further validated by results seen in Salama et al. (2023) [6], where hybrid approaches resulted in lower RMSE and MAE scores compared to stand-alone statistical models.

The hybrid CNN-LSTM model employed in this study was constructed to jointly leverage the feature extraction strength of Convolutional Neural Networks (CNNs) with the temporal memory capacity of Long Short-Term Memory (LSTM) units. The core motivation was to enable efficient learning across both short-term seasonal patterns and long-range temporal dependencies intrinsic to water demand data.

The input to the model consisted of multivariate sequences with shape (T, d) , where T represents the time steps (e.g., 336 for 2 weeks) and d the number of engineered features per time step. The first stage of the model applied a series of 1D convolutional layers with kernel size $k = 3$ and padding set to preserve sequence length. This acted as temporal filters that scanned across the input series to extract local trends and high-frequency variations. To preserve the temporal alignment of features with their original time indices, the output of the convolutional block was reshaped and passed directly into an LSTM layer without pooling. The LSTM decoder, consisting of 64 hidden units, captured the sequential dependencies across the latent feature vectors from the CNN. The final hidden state from the LSTM was projected through a fully connected dense layer with linear activation to produce the target sequence of size H , the hourly predictions.

4.3 Transformer

Transformers are a specialised type of neural network architecture, originally introduced by Vaswani in their 2017 paper 'Attention is All You Need' [7], containing large encoder-decoder blocks for processing data; expanding the wide range of capabilities in machine learning. Their powerful functionality in sequential modelling tasks in natural language processing, DNA sequencing and time series forecasting stems from their ability to capture long-range dependencies through self attention without relying on recurrence. Unlike its traditional predecessors that struggle to retain information over long sequences with either vanishing gradients or limited memory windows, the Transformer overcomes this with its core self-attention component. This computes a weighted representation of each input element based on its relationship to every other element in the sequence, allowing the model to process the entire input simultaneously. Formally, given an input sequence $\mathbf{X} \in \mathbb{R}^{L \times d_{\text{model}}}$, where L is the sequence length and d_{model} is the model dimensionality, the Transformer projects $\mathbf{X}$ into three separate representations: queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$, defined as:

$$\begin{aligned}\mathbf{Q} &= \mathbf{X}\mathbf{W}_Q, \\ \mathbf{K} &= \mathbf{X}\mathbf{W}_K, \\ \mathbf{V} &= \mathbf{X}\mathbf{W}_V,\end{aligned}$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learnable parameter matrices and d_k is the dimensionality of the attention subspace. These projections are used to compute attention weights through the scaled dot-product attention mechanism:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

The dot product $\mathbf{Q}\mathbf{K}^\top$ captures similarity between each query and key pair, while the division by $\sqrt{d_k}$ prevents the resulting values from growing too large, which would otherwise cause softmax to produce extremely small gradients. The output is a weighted sum of value vectors, with weights indicating the relevance of each position to the current token. Parallelism is therefore enabled through this transformation of input sequences, which are combined through matrix operations to compute attention scores. Each position attends to all others so the model evaluates contextual relevance across the entire sequence in a single self-attention layer, achieving context-aware representation at each time step and capturing complex, non-linear relationships between elements regardless of their position. This ability to model full-sequence context in one single layer not only enhances the model's representational capacity but also eliminates the sequential bottlenecks inherent in recurrent architectures.

Since the Transformer lacks a convolutional structure, it doesn't inherently encode the order of elements within a sequence, positional encoding is added to the input embeddings at the base of both the encoder and decoder stacks. Following from the paper, I used sinusoidal positional encoding which uses a series of sine and cosine functions at varying frequencies, producing unique, continuous encodings for each position.

Recent literature has highlighted the Transformer's impact on time series applications with Transformers, parallel computation, and logarithmic depth' [8] providing a comprehensive study and survey. Their review categorises Transformer-based models in time series into the following function dimensions: input representation, attention mechanism design, architecture enhancements, and forecasting objectives. This taxonomy pushes forward the challenges posed by time series data which are not fully addressed by the original Transformer framework such as long term dependencies, non-stationarity and multi-scale dynamics. One of the paper's key observations is that despite vanilla Transformers' theoretical ability to model global context, their practicality may be unsuitable

for time series forecasting due to computational inefficiencies and space complexity of the self-attention mechanism becoming a bottleneck in long horizon forecasting tasks. Furthermore, standard Transformers often-times lack inductive biases that make them prone to overfitting and are not inherently autoregressive, meaning they require additional architectural components or decoding schemes to operate in a step-wise forecasting fashion. These practical challenges have catalysed research into more efficient transformer variants that are specifically tailored for time series data.

4.4 Informer

Several notable architectural innovations have emerged to adapt the Transformer framework in long sequential forecasting. Among these, is the Informer introduced by Zhou et al. (2021) [9], which stands out as the key advancement. The Informer introduces two core contributions designed to overcome the limitations of the standard Transformer: ProbSparse self-attention and a distilling operation in the encoder. ProbSparse self-attention addresses the quadratic complexity of the attention mechanism by selectively focusing on the most ‘informative’ queries. This sparse attention mechanism significantly reduces time and memory complexity from $O(L^2)$ to $O(L \log L)$ where L is the sequence length, such that our model can process sequences of hundreds or thousands of time steps. In addition, the Informer improves generalisation through a self-attention distilling operation, which compresses the temporal dimension of the hidden states as they pass through the encoder layers. This hierarchical downsampling not only reduces redundant information but focuses on dominant temporal patterns present in water consumption. A combination of efficient attention and sequence distillation ensures high forecasting accuracy while drastically reducing computational overhead. Informer adopts a generative style decoder whilst the Transformer relies heavily on autoregressive decoding. Having a generative style decoder means that the model can predict the entire output sequence in one forward pass rather than step-by-step such that error accumulation over long periods can be reduced significantly and we can enable parallelised training. This design choice is more beneficial for multi-step forecasting water demand as my focus lays in stability and efficiency. Another key differentiating feature of the Informer is a log-sparse attention mask, where the top- u queries are selected through a statistical sparsity measurement.

$$\|\mathbf{q}_i \mathbf{K}^\top - \mu_i\|$$

where $\mu_i = \frac{1}{L} \sum_{j=1}^L \mathbf{q}_i \cdot \mathbf{k}_j$ is the mean dot-product attention score across keys for query $\mathbf{q}_i$. Only queries with the largest deviation from their mean contribute to the final attention distribution. The core multi-head attention functionality, where parallel attention heads are computed and concatenated permits the model to jointly attend to information from different representation subspaces. This log-sparse attention mechanism enables the model to scale efficiently to long input sequences without incurring the prohibitive memory costs typically associated with standard attention. By attending only to the most “informative” queries—those with the highest deviation from the average attention score. Informers effectively reduce redundancy in the attention map, focusing computation on inputs with the greatest predictive utility.

In implementing the Informer architecture for multi-step water demand forecasting, several practical modifications and simplifications were introduced to adapt my model to the characteristics of my data. My architecture retained the full encoder-decoder; the encoder processing the historical time series sequence using stacked attention blocks. Each encoder layer consisted of full multi-head attention mechanism, followed by position-wise feedforward layers and residual connections.

The decoder, in turn, received a masked version of the future sequence (typically a sequence of zeros for the DI target values) along with known covariates such as hour-of-day and day-of-week features for the forecast window. This decoder employed attention blocks, but crucially, incorporated both self-attention (within the decoder inputs) and cross-attention (to the encoder outputs), allowing the model to align future demand with historical trends. My encoder also included hierarchical downsampling after select layers to reduce the temporal dimension of hidden states as the sequence progressed through deeper layers. This strategy helped the model to abstract away fine-grained noise in favour of behavioural patterns and was not applied to the decoder, because it needs to retain temporal resolution for accurate forecasts. The decoder produces the final output by projecting its last layer’s hidden states through a fully connected layer to match the target horizon. Since the model is non-autoregressive, the entire forecast sequence is generated in one forward pass. This approach enhances stability during training and avoids cumulative error propagation, which is a common drawback in autoregressive decoding schemes.

To train the Informer on real-world water demand sequences, I adopted a sliding window approach for transforming the raw time series into supervised learning format, which I adopted for each of my models. Specifically, a fixed-length input window (e.g., 96 hourly values) was used to predict the next fixed-length output horizon (e.g., 168 hours). These windows were shifted forward through the data with a defined stride, generating overlapping

examples that expose the model to a rich variety of temporal contexts. This framing encourages the model to generalise across different weeks, seasons, and demand regimes, as it repeatedly encounters a diverse range of input-output mappings during training. Moreover, this method supports the model’s non-autoregressive forecasting framework by providing clearly defined past and future segments for each prediction task.

5 Bayesian Inference

Traditional forecasting methods often report uncertainty using confidence intervals derived from frequentist assumptions—such as Gaussian residuals or repeated sampling over training subsets, however such approaches often underestimate the true uncertainty. For all industrial sectors, the future is uncertain and so any forecast produced by our models is essentially a conditional probability statement with the conditioning being our existing state of knowledge [10]. This epistemic uncertainty is best captured within the Bayesian framework, where forecasts are viewed not as single-point predictions but as full probability distributions reflecting degrees of belief given current evidence. In a time series context, this involves constructing dynamic models that evolve with time, allowing parameters themselves to change as new data becomes available. Bayesian forecasting embraces the principle of management by exception: while routine periods rely on model-based forecasts, deviations—caused by events not included in the model—are handled by subjective intervention, allowing external information and expert judgement to adjust the model outputs.

Unlike static models that assume fixed relationships over time, Bayesian dynamic models acknowledge that information decays and past observations may become less relevant due to evolving conditions. This temporal discounting is inherent to dynamic linear models (DLMs), which adjust parameter estimates dynamically and prioritize recent data. Such adaptability provides a robust approach to forecasting under uncertainty, where both the model structure and the beliefs it encodes can be continuously refined. Recent developments of deep learning models have shown significant promise in time series forecasting due to their flexibility in capturing complex non-linear patterns, yet these models typically lack well-calibrated uncertainty estimates - an essential tool for strategic decision-making.

One of the foundational approaches to Bayesian deep learning is Variational Inference (VI), a method used to approximate complex posterior distributions over model parameters. Since computing the exact posterior $p(\theta \mid D)$ is generally intractable due to high-dimensional integrals over parameters, VI introduces a family of simpler, tractable distributions $q(\theta)$ and optimizes them to be as close as possible to the true posterior. This is done by minimising Kullback-Leibler (KL) divergence between $p(\theta \mid D)$ and $q(\theta)$. In the context of deep learning, this involves training the network not to learn a fixed set of weights, but to learn a distribution over weights. As a result, predictions from the model can reflect uncertainty over those weights by sampling from $q(\theta)$ during inference.

To make variational inference more scalable and accessible within deep learning frameworks, Monte Carlo (MC) -Dropout has emerged as a practical and scalable method for approximating Bayesian inference in deep neural networks [11]. MC Dropout interprets dropout—a common regularization technique used during training—as a form of approximate variational inference in a deep Gaussian process. By enabling dropout not only during training but also at test time, the model effectively samples from a distribution over its parameters. Repeated stochastic forward passes through the network then yield a predictive distribution, from which both point estimates and credible intervals can be derived. Standard neural models such as LSTMs or Transformers provide deterministic outputs that may overfit noisy training data and fail to convey the uncertainty inherent in future predictions. MC Dropout enables these models to express epistemic uncertainty—uncertainty about the model parameters themselves—by approximating the posterior distribution over weights, which is extremely beneficial to decision-makers to understand not just the projected forecast, but also the confidence associated with each prediction. MC Dropout does not require explicitly defining a prior or posterior distribution in closed form. Instead, it uses a stochastic regularization mechanism already available in most deep learning frameworks, making it computationally attractive and easy to implement.

Applying Monte Carlo Dropout as a variational approximation method within a deep learning framework enables the quantification of predictive uncertainty when forecasting water demand. In particular, the model outputs a sequence of future water demands $\hat{\mathbf{y}} \in \mathbb{R}^H$ where H is the dynamic forecast horizon, given historical inputs $\mathbf{x}_{\text{enc}}$ and corresponding decoder inputs $\mathbf{x}_{\text{dec}}$.

Let $f(\cdot; \theta)$ denote the neural network model with parameters θ . Instead of producing a single deterministic forecast, we estimate a *predictive distribution* over outputs by performing multiple stochastic forward passes through the model with dropout enabled:

$$p(\hat{\mathbf{y}} \mid \mathbf{x}_{\text{enc}}, \mathbf{x}_{\text{dec}}, \mathcal{D}) = \int p(\hat{\mathbf{y}} \mid \mathbf{x}_{\text{enc}}, \mathbf{x}_{\text{dec}}, \theta) p(\theta \mid \mathcal{D}) d\theta \quad (2)$$

Since computing the exact posterior $p(\boldsymbol{\theta} \mid \mathcal{D})$ is intractable, we use Monte Carlo (MC) Dropout to approximate the integral by averaging T stochastic forward passes:

$$\hat{\mathbf{y}} \approx \frac{1}{T} \sum_{t=1}^T f(\mathbf{x}_{\text{enc}}, \mathbf{x}_{\text{dec}}; \boldsymbol{\theta}_t) \quad (3)$$

We also compute the sample variance for quantifying epistemic uncertainty:

$$\mathbb{V}[\hat{\mathbf{y}}] \approx \frac{1}{T} \sum_{t=1}^T f(\mathbf{x}_{\text{enc}}, \mathbf{x}_{\text{dec}}; \boldsymbol{\theta}_t)^2 - \left(\frac{1}{T} \sum_{t=1}^T f(\mathbf{x}_{\text{enc}}, \mathbf{x}_{\text{dec}}; \boldsymbol{\theta}_t) \right)^2 \quad (4)$$

For instance, sharp increases in uncertainty may indicate anomalous conditions such as unusual weather events, demand spikes, or operational changes in the network. By explicitly modelling uncertainty, MC Dropout helps prevent overconfident predictions that could mislead resource allocation or supply planning. From the predictive distribution, we can construct credible intervals. Assuming approximate normality of the outputs due to the Central Limit Theorem (as T increases), a 95% confidence interval for each forecasted timestep can be constructed as:

$$\hat{\mathbf{y}} \pm 1.96 \cdot \sqrt{\mathbb{V}[\hat{\mathbf{y}}]} \quad (5)$$

This allows us to visualise not just the central forecast trajectory, but also a band of uncertainty surrounding it—critical for operational robustness under uncertainty. In my implementation, I ran $T = 50$ stochastic forward passes using the `training=True` mode during inference to preserve dropout behaviour.

Stochastic Weight Averaging-Gaussian (SWAG) approximates a Bayesian posterior over model parameters by leveraging the idea that the trajectory of weights during training contains valuable information about the underlying uncertainty. Instead of learning a distribution via dropout or reparameterized variational layers, SWAG directly constructs a Gaussian approximation to the posterior distribution over weights using snapshots collected from the stochastic optimization path of stochastic gradient descent. However, its implementation posed significant challenges primarily because it demands access to the full stochastic training trajectory which has to be sampled and stored during training. Although SWAG has provided well-calibrated predictive uncertainty in classification tasks, its use in time series remains for the most part under-explored with studies highlighting SWAG’s limitation being computationally expensive and impractical for long-scale sequence tasks.

A computationally efficient alternative to full Bayesian neural networks is to apply Bayesian inference exclusively to the final (output) layer of a trained model. This approach, often referred to as Bayesian last-layer inference, approximates the posterior distribution over the weights of the output layer while keeping the rest of the network deterministic. Kristiadi et al. (2020) [12] demonstrate that this strategy can substantially improve uncertainty calibration without requiring costly modifications to the overall model architecture. Following training, the final layer’s parameters—typically a fully connected projection from the final hidden representation to the forecast output—are treated as random variables rather than fixed point estimates. A Laplace approximation is then employed to approximate the posterior distribution over these weights with a multivariate Gaussian, centred at the maximum a posteriori (MAP) estimate and with a covariance matrix derived from the inverse of the Hessian of the loss function evaluated at the MAP solution.

If $\mathbf{w}$ denotes the weights of the final layer and $\mathcal{L}(\mathbf{w})$ is the negative log-likelihood (or training loss), the posterior is approximated as:

$$p(\mathbf{w} \mid \mathcal{D}) \approx \mathcal{N}(\mathbf{w}_{\text{MAP}}, \mathbf{H}^{-1}),$$

where $\mathbf{H} = \nabla^2 \mathcal{L}(\mathbf{w})$ is the Hessian matrix of second derivatives with respect to the weights. In practice, $\mathbf{H}$ is often approximated using either the diagonal or a low-rank variant to ensure tractability in high dimensions. At inference, predictions are obtained by sampling weight vectors from this Gaussian posterior and averaging the resulting outputs, thereby producing a predictive distribution.

This approach shares philosophical similarities with MC Dropout, in that both aim to approximate epistemic uncertainty through variational means. However, Bayesian last-layer inference confines the stochasticity to the final projection head, which significantly reduces both memory and computational overhead compared to applying dropout across the full model. Despite its simplicity, this method has been shown to yield well-calibrated uncertainty estimates, making it particularly attractive for large-scale deep models such as the Informer, where full Bayesian inference would be computationally prohibitive.

Another Bayesian technique explored in this study was Hierarchical Stochastic Attention (HSA) which introduces stochasticity directly into the Transformer’s attention mechanism. Unlike MC Dropout which injects

randomness globally through dropout activations, HSA operates by sampling attention weights or scores during each forward pass. Taking the standard attention formula:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

HSA perturbs the attention scores stochastically during inference. One approach is by treating attention logits as random variables:

$$\mathbf{A} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2) \quad (6)$$

This results in sampled attention maps across multiple forward passes, each representing a different plausible set of alignments between queries and keys.

6 Results

For evaluating my models performance on water demand forecasting, I used three standard regression metrics to capture differing aspects of predictive accuracy: Mean Absolute Error (MAE), Mean Squared Error (MSE) and the coefficient of determination, R^2 .

- **Mean Absolute Error (MAE)** provides an intuitive average of the absolute differences between predicted and actual values, expressed in the same units as the target variable:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Mean Squared Error (MSE)** penalises larger errors more heavily due to the squaring term, making it more sensitive to outliers:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Coefficient of Determination (R^2)** measures the proportion of variance in the observed data that is explained by the model:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

These metrics were computed on a withheld test set using overlapping forecast windows. Given the operational implications of both over- and under-prediction in water supply, both absolute and squared-error metrics were used in parallel to ensure robust model assessment.

Table 1: Model performance for 1, 4, and 8 week forecasts.

Model & Horizon	MAE	MSE	R^2
CNN-LSTM 1w	2.82	23.63	0.848
CNN-LSTM 4w	2.79	23.39	0.850
CNN-LSTM 8w	3.12	26.47	0.831
Transformer 1w	2.81	27.09	0.845
Transformer 4w	1.09	2.56	0.985
Transformer 8w	1.12	2.78	0.985
Informer 1w	0.688	0.85	0.99
Informer 4w	0.860	1.00	0.99
Informer 8w	0.863	1.08	0.99

At first when I began designing my LSTM model, I was encouraged by its performance — my first model achieved a high R^2 score and visually aligned forecasts over multiple steps ahead of one week. The architecture was relatively simple: a unidirectional LSTM trained on sliding input windows of historical Distribution Input (DI), combined with engineered cyclical time features (e.g., sine and cosine encodings of hour-of-day and day-of-week). However upon revisiting my model’s code I saw that it was leveraging future-derived information

during inference in a way that compromised the integrity of its predictions. Specifically, during the recursive multi-step forecasting phase, the model was repeatedly supplied with analytically generated time features corresponding to actual future timestamps. Whilst this does not involve leakage of future DI values, it does allow the model to infer strong periodic cues — for example, that it is forecasting at a specific time — which may lead it to output typical DI values for that time regardless of the input sequence. As a result, the model may have appeared to forecast accurately not because it captured temporal dependencies from the DI signal itself, but because it relied heavily on static, repetitive patterns embedded in the calendar. This shortcut does not invalidate the model outright, but it does inflate its apparent performance and limits its robustness under dynamic or irregular demand conditions. Moving forward, it became necessary to adapt my original architecture.

Despite the overall challenges encountered when training LSTM-based models for water demand forecasting—particularly the emergence of negative R^2 scores in more recent iterations—I explored a post-hoc adjustment technique to improve the forecast trajectories by embedding domain-aware priors into the output. The practical benefits of the anomaly-aware post-filter in correcting misaligned forecasts captured some abrupt DI shifts yet it often failed to produce consistent improvements across the test set. This was partly due to the filter operating as a rule-based overlay rather than influencing the model’s internal dynamics. As a result, I sought to improve model generalisation by reconfiguring the architecture itself. I implemented a hybrid LSTM-CNN model, leveraging convolutional layers for local temporal feature extraction followed by LSTM layers for sequential modelling. The convolutional layers acted as adaptive filters that captured short-term fluctuation patterns in DI sequences—such as hourly volatility and transient demand spikes—before passing the learned features into LSTM layers capable of retaining broader temporal dependencies. This architectural fusion allowed the model to respond more robustly to both gradual seasonal trends and abrupt localised changes, outperforming earlier LSTM-only models in terms of both visual forecast alignment and generalisation under operational variability. One important strength of this model lies in its ability to decouple spatial and temporal feature extraction, allowing the convolutional layers to abstract input representations before the LSTMs handle sequence modeling. This effectively reduces the sequence length the recurrent layers must process, alleviating the training bottlenecks typically associated with long input windows in vanilla LSTM and Transformer setups. The consistently strong performance—particularly at the 1-week and 4-week forecast intervals—suggests that the hybrid configuration is robust under varying temporal scales.

The Transformer model developed in my study diverged from the canonical “vanilla” Transformer by incorporating several architectural refinements for stabilising long-horizon forecasting. Unlike traditional encoder-decoder setups with positional encodings and multi-head attention blocks stacked uniformly, my implementation introduced separate encoder and decoder streams, each with independently learned positional encodings and multiple stacked attention layers interleaved with dense feed-forward blocks. This modular design was paired with residual connections and layer normalisation to preserve gradient flow and mitigate vanishing signal degradation across deep time steps. The Transformer achieved a strong performance at both 1-week and 4-week horizons, with minimal performance degradation ($\text{MAE} = 2.81 \rightarrow 1.09$, and $\text{MSE} = 27.09 \rightarrow 2.56$) suggesting that the model was not merely fitting short-term fluctuations but also learned to generalise over broader temporal structures. My most recent retraining of the 8-week Transformer model yielded highly positive results, contradicting the earlier observations of performance collapse, where both MAE and MSE were exponentially greater with even a R^2 score of 0.095. This turnaround is attributed to the redesigned decoder pipeline, which explicitly incorporated extended lookback sequences during training, thereby improving the model’s extrapolation capacity.

The Informer model achieved remarkably high predictive performance across all forecast horizons, yielding R^2 scores consistently at 0.99. This can be attributed to a combination of architectural choices and data characteristics. The model allowed time-based features to be fed into the encoder and decoder input. Whilst they don’t directly leak the target variable they encode strong periodic signals. Application of Monte Carlo dropout during inference served to stabilise predictions via ensemble averaging. The absence of ProbSparse attention — a key efficiency mechanism in the original Informer — suggests that the performance gains arose less from computational efficiency and more from architectural synergy and feature engineering. Nevertheless, these results must be interpreted cautiously, as the reliance on future-derived time features may limit generalisability under non-seasonal or anomalous conditions. Lastly, the use of Monte Carlo Dropout during inference provides Bayesian uncertainty estimates for smoothing out noise. While this technique adds robustness, the extremely narrow confidence intervals observed in the forecast plots suggest that the model remains highly confident, possibly due to the strong periodic signals in the input space.

A key determinant of my Informer’s performance is the configuration of the encoder and decoder context, defined by `seq_len` and `lab_len` (sequence length and label length). These hyperparameters govern how much historical information the model uses and how much past context is available during decoding, which directly

influence the learning capacity and forecast stability. A short sequence in the encoder input length may often fail to capture broad signals, whilst on the other hand overly long sequences can dilute any short-term trends visible and additionally introduce noise. In my early stages of experimentation, models trained with a short sequence of 4 days showed clear limitations in extrapolating long term structures, which resulted in flat predictions. This configuration became especially important when extending forecasts beyond a short horizon. For my 8-week experiment, I set `seq_len = 1008` and `label_len = 1344`, enabling the model to access sufficient historical context (6 weeks) while maintaining a long decoder context to learn temporal dependencies effectively.

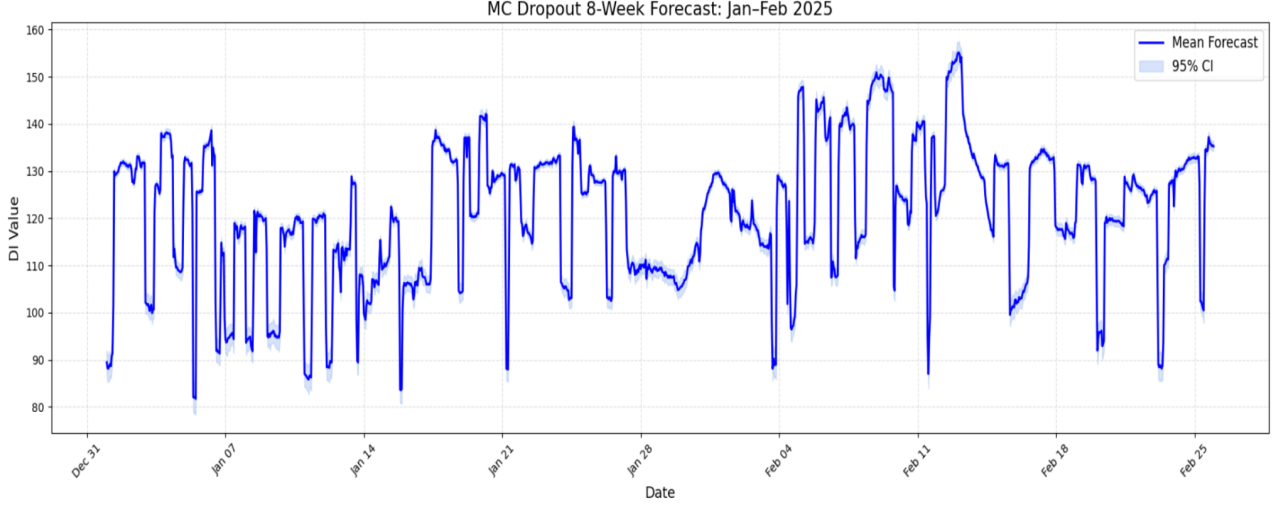


Figure 2: MC Dropout 8-Week Forecast for Restormel 10SRE01.

The residual plot below shows the difference between the actual DI values and the Informer model’s predicted means across the 8-week test horizon. Most of the residuals remain in a tight margin close to zero showing that the DI forecasts align very closely to the actual values. This tight clustering around the zero line is indicative of low-bias predictions and reflects the Informer’s strong ability to capture the underlying temporal dynamics of the water demand series. The deviations observed coincides with the sudden shifts seen in my DI data and the asymmetric distribution of errors being positive suggests that in most cases the Informer tends to underpredict. The model appears to not only generate accurate point predictions but does so with a degree of confidence that is corroborated by consistent residual behaviour. These results, when interpreted alongside the performance metrics and uncertainty plots, underscore the robustness of the Informer architecture in real-time water demand forecasting.

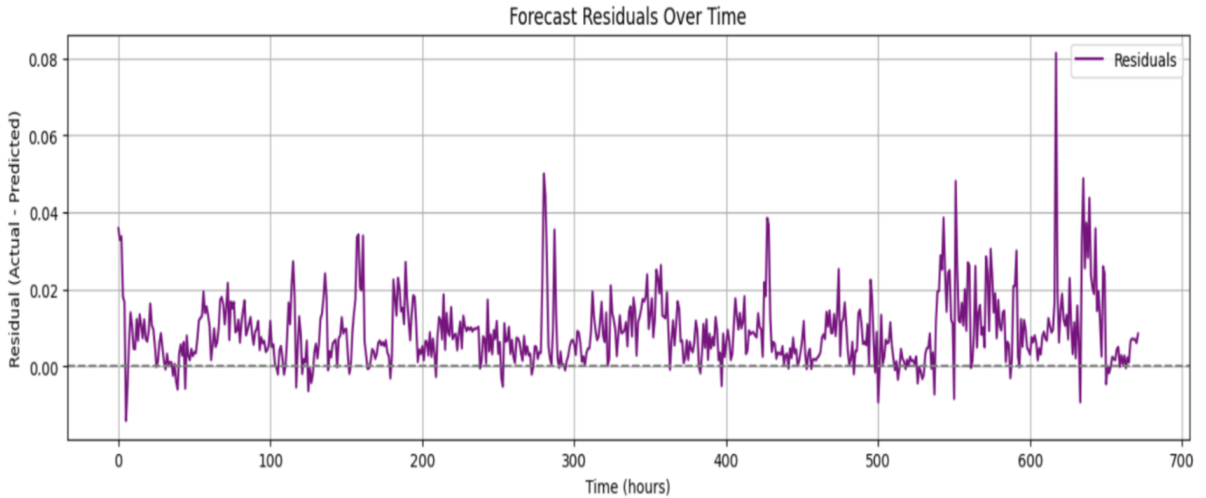


Figure 3: Residuals plot across the 8-week test length.

7 Future research

Alongside Transformer-based models, recent developments in deep time-series forecasting have introduced alternative architectures focused on either probabilistic forecasting or interpretability. One notable example is DeepAR (Salinas et al., 2020)[13], a probabilistic forecasting model developed by Amazon that leverages autoregressive recurrent neural networks (RNNs) to model the full predictive distribution of future values. DeepAR generates sample-based forecasts, allowing it to capture both aleatoric and epistemic uncertainty explicitly. Its design also supports multi-series learning, where information from related time series can be shared to improve generalisation—particularly useful for settings involving multiple water treatment works or district metered areas. DeepAR’s ability to generalise across entities (e.g., across multiple water zones or households) greatly enhances performance on individual series, especially when data is sparse. It also incorporates teacher forcing during training and ancestral sampling during inference, which ensures numerical stability and consistency when generating long-term forecasts. The model has been validated across industrial-scale datasets with millions of time series, demonstrating its robustness in production environments.

On the other hand, N-BEATS (Oreshkin et al., 2020)[14] was introduced as a deep learning architecture designed with interpretability and simplicity at its core. It is composed of fully connected blocks that learn basis expansions of the time series, explicitly decomposing signals into trend and seasonality components without the need for convolution, recurrence, or attention mechanisms. Its forecast is generated as the sum of forecasts from multiple such blocks, each of which is trained to either extrapolate trend or periodicity.

N-BEATS is considered reliable for several reasons. First, it avoids the risk of overfitting to short-term noise by leveraging a backcast–forecast architecture, where each block tries to reconstruct the input (backcast) while simultaneously producing a future prediction (forecast). This dual objective acts as a form of self-regularisation. Second, it demonstrates strong inductive biases toward decomposable structures common in real-world time series, such as those seen in energy consumption, retail sales, or water demand. Third, it outperforms several benchmarks even without access to known future covariates, making it robust in scenarios where exogenous information is limited or unreliable. Finally, its inherently interpretable structure—where each block’s output corresponds to a trend or seasonal component—offers transparency that is highly valuable in regulated environments or infrastructure planning settings. Both of these models represent high credible alternatives to Transformer-based forecasting approaches. Both models align with the intentions of my research with DeepAR excelling in multi-entity, uncertainty-aware forecasting tasks, whilst N-BEATS providing strong accuracy and transparency in univariate settings.

One promising avenue for future work lies in leveraging ensemble learning to further enhance the robustness and reliability of forecasts under uncertainty. Ensemble methods involve training multiple models—either homogeneous (same architecture with different initializations or training subsets) or heterogeneous (different architectures altogether)—and combining their outputs to form a final prediction. This technique is especially valuable in real-time water demand forecasting, where input distributions can shift due to seasonality, operational disruptions, or anomalous user behaviours. In ensemble forecasting, the combined output prediction $\hat{\mathbf{y}}$ is typically computed as the average (or weighted average) of predictions from M individual models:

$$\hat{\mathbf{y}} = \frac{1}{M} \sum_{m=1}^M f^{(m)}(\mathbf{x})$$

where $f^{(m)}(\mathbf{x})$ denotes the output of the m^{th} model given the input sequence $\mathbf{x}$. Each base learner may capture different aspects of the data distribution or generalize differently to unseen conditions, and aggregating their outputs can reduce variance and improve generalisation performance. In the case of probabilistic forecasting, where each base model returns a predictive distribution $p_m(\mathbf{y} \mid \mathbf{x})$, the ensemble’s output becomes a mixture model:

$$p(\mathbf{y} \mid \mathbf{x}) = \frac{1}{M} \sum_{m=1}^M p_m(\mathbf{y} \mid \mathbf{x})$$

This aggregated distribution is better calibrated and often has lower predictive uncertainty than individual models, as ensemble averaging tends to cancel out overconfident or underperforming components. When combined with MC Dropout or variational methods, ensemble techniques offer a strong foundation for Bayesian deep ensembles, where uncertainty is captured both through weight distributions and model diversity.

Since the accuracy and generalisability of demand forecasting models can be heavily influenced by external drivers, future exploration in the incorporation of exogenous variables, which include meteorological features—such as rainfall, temperature, humidity as well as socio-economic indicators like employment levels,

tourist footfall, and household occupancy rates. Such features are not part of the endogenous water demand signal but exert measurable influence over consumption patterns, particularly during seasonal transitions or atypical operational events. Integrating these exogenous drivers may help models disambiguate between stochastic noise and meaningful shifts in usage behaviour. For example, sharp drops in DI readings during cold spells or localised maintenance could be better interpreted with access to corresponding weather or scheduling metadata. Recent frameworks such as the Temporal Fusion Transformer (TFT) exemplify this design, explicitly embedding static metadata and time-varying exogenous variables to enhance long-horizon performance. Alternatively, attention-based models could dynamically re-weight the influence of different external features over time, enabling the model to focus more heavily on climate features during summer peaks or economic indicators during off-season variability.

Moreover, the inclusion of exogenous variables is particularly promising when combined with probabilistic or ensemble approaches. By constraining the model’s predictive space using known external drivers, epistemic uncertainty may be reduced, thereby improving both the confidence and calibration of forecasts. In practical terms, this could significantly enhance the operational value of water demand forecasts by making them more robust to non-stationary inputs and structural breaks in behaviour.

8 Ethical and Societal Implications

As machine learning models become increasingly implemented into essential infrastructure systems, it is vital to consider the broader ethical and societal implications of their deployment. Forecasting models rely on granular data collected from distributed sensors and smart meters. Although the data used in my study was aggregated at a WTW level, future iterations of similar models may try incorporate finer-grained customer-level data for modelling behavioural factors and smoother integration. This can introduce risks surrounding individual privacy, moreso if location-based or household-level consumption patterns are inferred. Ethical implementation must therefore comply with data protection regulations ensuring data anonymisation and clear consent frameworks where applicable.

Forecasting systems trained on historical data may inadvertently encode and perpetuate biases present in infrastructure investment. For example, regions with historically higher investment and more robust sensor coverage may receive disproportionately more accurate predictions, leading to inequitable service levels. Therefore, model performance should be benchmarked across diverse geographic and socio-economic regions, with fairness audits conducted regularly to ensure that predictive accuracy does not vary unreasonably across zones. While Bayesian models provide the advantage of uncertainty quantification, there remains a risk of over-reliance on algorithmic outputs in high-stakes decision-making. Forecast errors—especially under uncertainty—could lead to either over-provisioning or under-provisioning and so forecasts should be presented alongside interpretable explanations for decision makers to apply expert judgement, especially under abnormal conditions. Ultimately, embedding ethical considerations into the development and deployment of AI models in water systems is not merely a compliance exercise, but a prerequisite for building robust and socially responsible infrastructure.

9 Conclusion

My study contributes to the growing literature on sequential modelling’s capabilities specifically in industries that heavily rely on well-informed real-time decision making; demonstrating that lightweight Bayesian approximations, like MC Dropout, can be successfully integrated into long-horizon predictive models without sacrificing computational efficiency. Unlike many existing studies that prioritise point accuracy alone, this project emphasises uncertainty quantification as a core component of operational forecasting. Using Restormel WTW as a case study for my investigation into the effectiveness of deep learning and Bayesian inference on time series data provided empirical insights into the suitability of modern sequential models. Through systematic comparison of CNN-LSTM, Transformer, and Informer architectures, the Informer emerged as the most robust and accurate model across all forecasting horizons.

From an operational perspective, the ability to forecast hourly water demand over multi-week horizons with associated uncertainty intervals has substantial implications for utility planning and risk management. By framing forecasts probabilistically rather than deterministically, operators can better assess the likelihood of over or under supply and make informed decisions regarding treatment scheduling, energy usage, and system resilience. The application of MC Dropout during inference enabled the generation of predictive distributions through repeated stochastic forward passes, offering a scalable and computationally efficient approximation to Bayesian inference that is well suited to real-time deployment.

However, the results also highlight important limitations that must be acknowledged. The strong performance of the Informer was partly driven by the inclusion of calendar-based covariates, such as hour-of-day and day-of-week encodings, which introduce strong periodic structure into the input space. While effective under stable operating conditions, this reliance may reduce robustness during anomalous events where historical periodicity breaks down, such as extreme weather, infrastructure outages, or abrupt behavioural changes. Furthermore, the consistently narrow uncertainty bands observed in several forecasting scenarios suggest potential overconfidence in highly structured data regimes, reinforcing the need for more expressive uncertainty mechanisms or hybrid Bayesian approaches.

Beyond predictive accuracy, practical deployment of such models requires careful consideration of interpretability and system integration. Forecasting tools must interface seamlessly with existing SCADA systems, handle streaming data reliably, and present outputs in a form that supports human decision-making. Techniques such as attention-weight analysis or post-hoc explainability methods may provide valuable insights into which historical patterns or features most influence forecasts, thereby increasing trust and adoption among domain experts.

In conclusion, this study demonstrates that combining Transformer-based architectures with Bayesian inference offers a compelling and practical framework for uncertainty-aware water demand forecasting. The Informer’s strong performance across extended horizons, paired with probabilistic forecasting via MC Dropout, represents a meaningful step toward more resilient and transparent digital water management systems. As water utilities increasingly transition toward data-centric operation, such models have the potential to play a central role in enabling adaptive, efficient, and risk-aware infrastructure planning in the face of growing uncertainty.

References

- [1] Department for Environment, Food & Rural Affairs. Plan for water: Our integrated plan for delivering clean and plentiful water. <https://www.gov.uk/government/publications/plan-for-water-our-integrated-plan-for-delivering-clean-and-plentiful-water/plan-for-water-our-integrated-plan-for-delivering-clean-and-plentiful-water>, April 2023. Accessed: 2025-06-03.
- [2] Chris Holt and Simon Davis. Restormel wtw: Gac pesticide control and capital maintenance project. *UK Water Projects*, 2015. South West Water AMP5 capital programme overview.
- [3] A. Gil-Gamboa, P. Paneque, O. Trull, and A. Troncoso. Medium-term water consumption forecasting based on deep neural networks. *Expert Systems with Applications*, 247:123234, 2024.
- [4] Kleantith Malialis, Nefeli Mavri, Stelios G. Vrachimis, Marios S. Kyriakou, Demetrios G. Eliades, and Marios M. Polycarpou. Urban water consumption forecasting using deep learning and correlated district metered areas. arXiv preprint arXiv:2501.00158, 2024.
- [5] Enrico Creaco, Yujie Yuan, Dragan A. Savić, and Giuseppe Becciu. Household water demand forecast at thames water: Application of machine learning models to smart meter data. *Journal of Hydroinformatics*, 25(2):275–289, 2023.
- [6] Takashi Kuremoto, Kenji Kobayashi, and Masaru Obayashi. Hybrid forecasting methods for power load forecasting. *Electronics*, 12(9):2019, 2023.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [8] Ying Zhang, Yue Wu, Zhenhua Xu, Xue Zuo, Xiang Liu, Yiqing Lin, Yanchi Liu, and Junchi Xu. A survey on transformer-based models in time series applications. *arXiv preprint arXiv:2402.09268*, 2024.
- [9] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11106–11115, 2021.
- [10] Andy Pole, Mike West, and Jeff Harrison. *Applied Bayesian Forecasting and Time Series Analysis*. Chapman and Hall/CRC, 1st edition, 1994.
- [11] Yarín Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48, pages 1050–1059. PMLR, 2016.

- [12] Alexius Kristiadi, Matthias Hein, and Philipp Hennig. Being bayesian, even just a bit, fixes overconfidence in relu networks. *arXiv preprint arXiv:2002.10118*, 2020.
- [13] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. Deepar: Probabilistic forecasting with autoregressive recurrent networks. In *International Journal of Forecasting*, volume 36, pages 1181–1191. Elsevier, 2020.
- [14] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-beats: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2020.